

Helwan University
Faculty of Computer Science and Artificial Intelligence

Bibliotheca

Library Management System

Software Requirements Specification (SRS)

Prepared by:

Radwa Meshref

Sondos Ahmed

Aya Taha

Mostafa Yasser

Mohamed Mansour

Marwan Abdelfattah

Software Engineering 2 — Spring 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for Bibliotheca, a Library Management System developed using a microservices architecture. The system enables libraries to manage books, categories, user accounts, and borrowing operations through a modern web-based interface.

1.2 Project Scope

Bibliotheca is a full-stack web application that provides:

- A React-based frontend for user interaction
- A Spring Boot microservices backend with REST APIs
- JWT-based authentication and role-based access control
- Microservices architecture with Spring Cloud and Netflix Eureka
- An API Gateway for centralized routing
- Aspect-Oriented Programming (AOP) for cross-cutting concerns

1.3 Definitions and Acronyms

Term	Definition
API	Application Programming Interface
AOP	Aspect-Oriented Programming — a programming paradigm that separates cross-cutting concerns
JWT	JSON Web Token — a compact, URL-safe means of representing claims for authentication
REST	Representational State Transfer — architectural style for distributed systems
SRS	Software Requirements Specification
Microservices	An architectural style that structures an application as a collection of small, independent services
Eureka	Netflix service discovery server used to register and locate microservices
API Gateway	A single entry point that routes requests to the appropriate microservice
CORS	Cross-Origin Resource Sharing — a mechanism for browser-server communication
BORROWER	A user role that can browse and borrow books

LIBRARIAN	A user role that can manage books and view borrow records
ADMIN	A user role with full system access including user and category management

2. System Overview

2.1 System Description

Bibliotheca is a distributed Library Management System built on a microservices architecture. The system consists of five independent backend services coordinated through a centralized API Gateway and registered with a Netflix Eureka service discovery server. Users interact with the system through a responsive React frontend application.

2.2 System Architecture

The system follows a microservices architecture pattern with the following components:

Component	Technology	Port	Responsibility
Eureka Server	Spring Cloud Netflix	8761	Service discovery and registration
API Gateway	Spring Cloud Gateway (WebFlux)	8080	Centralized routing, CORS handling
Auth Service	Spring Boot 3.2.5	8081	User registration, login, JWT generation, AOP logging
Catalog Service	Spring Boot 3.5.x	8084	Category management
Book Service	Spring Boot 3.2.5	8083	Book CRUD operations
Borrow Service	Spring Boot 3.2.5	8082	Borrowing and returning books
React Frontend	React.js	3000	User interface for all system operations

3. Functional Requirements

3.1 User Authentication Module (Auth Service)

ID	Requirement	Priority
FR-01	The system shall allow new users to register with a username, password, and role	High
FR-02	The system shall authenticate users via username and password	High
FR-03	The system shall generate a JWT token upon successful login	High
FR-04	The system shall validate JWT tokens on every protected API request	High
FR-05	The system shall reject requests with invalid or expired JWT tokens	High
FR-06	The system shall support three user roles: ADMIN, LIBRARIAN, and USER	High

3.2 Book Management Module (Book Service)

ID	Requirement	Priority
FR-07	The system shall allow ADMIN and LIBRARIAN to add new books	High
FR-08	The system shall allow all authenticated users to view all books	High
FR-09	The system shall allow retrieval of a single book by its ID	High
FR-10	The system shall allow ADMIN and LIBRARIAN to update book details	High
FR-11	The system shall allow ADMIN and LIBRARIAN to delete books	High

FR-12	The system shall track available copies and total quantity for each book	High
FR-13	The system shall automatically decrement available copies when a book is borrowed	High
FR-14	The system shall automatically increment available copies when a book is returned	High

3.3 Category Management Module (Catalog Service)

ID	Requirement	Priority
FR-15	The system shall allow ADMIN to create new book categories	Medium
FR-16	The system shall allow all authenticated users to view all categories	Medium
FR-17	The system shall allow ADMIN to update existing categories	Medium
FR-18	The system shall allow ADMIN to delete categories	Medium

3.4 Borrowing Module (Borrow Service)

ID	Requirement	Priority
FR-19	The system shall allow authenticated users to borrow available books	High
FR-20	The system shall set a due date of 7 days from the borrow date	High
FR-21	The system shall prevent borrowing when no copies are available	High
FR-22	The system shall allow users to return borrowed books	High

FR-23	The system shall record the return date upon book return	High
FR-24	The system shall allow ADMIN and LIBRARIAN to view all borrowing records	High
FR-25	The system shall record the username of the borrower for each borrowing record	High

4. Non-Functional Requirements

4.1 Security

ID	Requirement
NFR-01	All API endpoints except login and register shall require JWT authentication
NFR-02	Passwords shall be encrypted using BCrypt hashing algorithm
NFR-03	JWT tokens shall expire after 24 hours
NFR-04	CORS shall be configured to allow requests only from the frontend origin (localhost:3000)
NFR-05	Role-based access control shall restrict sensitive operations to authorized roles only

4.2 Performance

ID	Requirement
NFR-06	The system shall respond to API requests within 2 seconds under normal load
NFR-07	The API Gateway shall route requests to the correct service without introducing significant latency
NFR-08	The system shall support concurrent users without performance degradation

4.3 Scalability and Maintainability

ID	Requirement
NFR-09	Each microservice shall be independently deployable and scalable
NFR-10	The system shall use Netflix Eureka for dynamic service discovery
NFR-11	The system shall be fully Dockerized for consistent deployment across environments
NFR-12	Each service shall maintain its own independent database

5. User Roles and Permissions

The system implements three distinct user roles, each with a specific set of permissions:

Feature	ADMIN	LIBRARIAN	USER (BORROWER)
Register / Login	Yes	Yes	Yes
View Books	Yes	Yes	Yes
Add / Edit / Delete Books	Yes	Yes	No
View Categories	Yes	Yes	Yes
Manage Categories	Yes	No	No
Borrow Books	Yes	Yes	Yes
Return Books	Yes	Yes	Yes
View All Borrow Records	Yes	Yes	No
View My Borrowed Books	Yes	Yes	Yes
Manage Users	Yes	No	No
Dashboard Access	Yes	No	No

6. Object Constraint Language (OCL)

Constraint ID	Context	OCL Expression	Description
OCL-01	User	<code>self.isRegistered = true</code>	A user must be registered before performing any actions in the system
OCL-02	User	<code>self.password = self.confirmPassword</code>	Password and confirm password must be identical during registration
OCL-03	Book	<code>self.availableCopies = 0 implies self.isAvailable = false</code>	If no copies are available, the book becomes unavailable
OCL-04	BorrowingRecord	<code>self.returned = true implies self.returnDate <> null</code>	If a book is returned, return date must be set
OCL-05	BorrowingRecord	<code>self.borrowDate < self.dueDate</code>	Borrow date must always be before due date
OCL-06	BorrowingRecord	<code>self.user.borrowingRecords->forAll(br br.book <> self.book or br.returned = true)</code>	User cannot borrow the same book again unless the previous borrowing is returned
OCL-07	User, Book	<code>self.returned = true implies self.availableCopies = self.availableCopies + 1</code>	When a book is returned, available copies increase
OCL-08	User, Book	<code>self.borrowedBy->notEmpty() implies self.availableCopies = self.availableCopies - 1</code>	When a book is borrowed, available copies decrease

7. Microservices Architecture

7.1 Service Communication

All client requests are routed through the API Gateway running on port 8080. The gateway forwards requests to the appropriate microservice based on the URL path. Service discovery is handled by Netflix Eureka, allowing services to locate each other dynamically without hardcoded URLs.

Path Pattern	Target Service	Port
/users/**	Auth Service	8081
/borrow/**	Borrow Service	8082
/categories/**	Catalog Service	8084
/books/**	Book Service	8083

7.2 Aspect-Oriented Programming (AOP)

The Auth Service implements AOP for cross-cutting concerns. A `LoggingAspect` class uses the `@Around` advice to intercept all controller method calls, automatically logging:

- The class and method name being executed
- The execution time in milliseconds
- Entry and exit points for each API call

This approach eliminates the need to add logging code manually to each controller method, following the DRY (Don't Repeat Yourself) principle. The pointcut targets all methods in the `auth_service.controller` package.

7.3 Inter-Service Communication

The Borrow Service communicates with the Book Service using a Feign Client (`CatalogClient`) to verify book availability before processing a borrowing request. This follows the microservices pattern of service-to-service communication via REST APIs.

8. Database Design

Each microservice maintains its own independent database, following the database-per-service pattern of microservices architecture.

8.1 Auth Service Database — users table

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(255)	NOT NULL, UNIQUE	User's login name
password	VARCHAR(255)	NOT NULL	BCrypt-hashed password
role	VARCHAR(50)	NOT NULL	User role: ADMIN, LIBRARIAN, or USER

8.2 Book Service Database — books table

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique book identifier
title	VARCHAR(255)	NOT NULL	Title of the book
author	VARCHAR(255)		Author of the book
pages	INT		Number of pages
available	BOOLEAN		Whether the book is available
quantity	INT	DEFAULT 1	Total number of copies
available_copies	INT	DEFAULT 1	Current available copies
category_id	BIGINT		Foreign key reference to category

8.3 Catalog Service Database — categories table

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique category identifier
name	VARCHAR(255)	NOT NULL	Category name (e.g., Fiction, Science)
description	VARCHAR(500)		Optional description of the category

8.4 Borrow Service Database — borrowing_records table

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique record identifier
user_id	BIGINT		ID of the user who borrowed the book
book_id	BIGINT		ID of the borrowed book
username	VARCHAR(255)		Username of the borrower
borrow_date	DATE		Date when the book was borrowed
due_date	DATE		Date when the book should be returned
return_date	DATE		Actual date of return (null if not returned)
returned	BOOLEAN	DEFAULT false	Whether the book has been returned

9. Docker Deployment

The entire system is containerized using Docker, enabling consistent deployment across different environments. Each microservice has its own Dockerfile, and Docker Compose orchestrates all services together.

9.1 Docker Containers

Container	Image	Port Mapping	Description
eureka-server	library/eureka-server	8761:8761	Service discovery server
api-gateway	library/api-gateway	8080:8080	API Gateway and routing
auth-service	library/auth-service	8081:8081	Authentication and user management
catalog-service	library/catalog-service	8084:8084	Category management
book-service	library/book-service	8083:8083	Book management
borrow-service	library/borrow-service	8082:8082	Borrowing management
mysql	mysql:8.0	3306:3306	MySQL database server
frontend	library/frontend	3000:3000	React frontend application

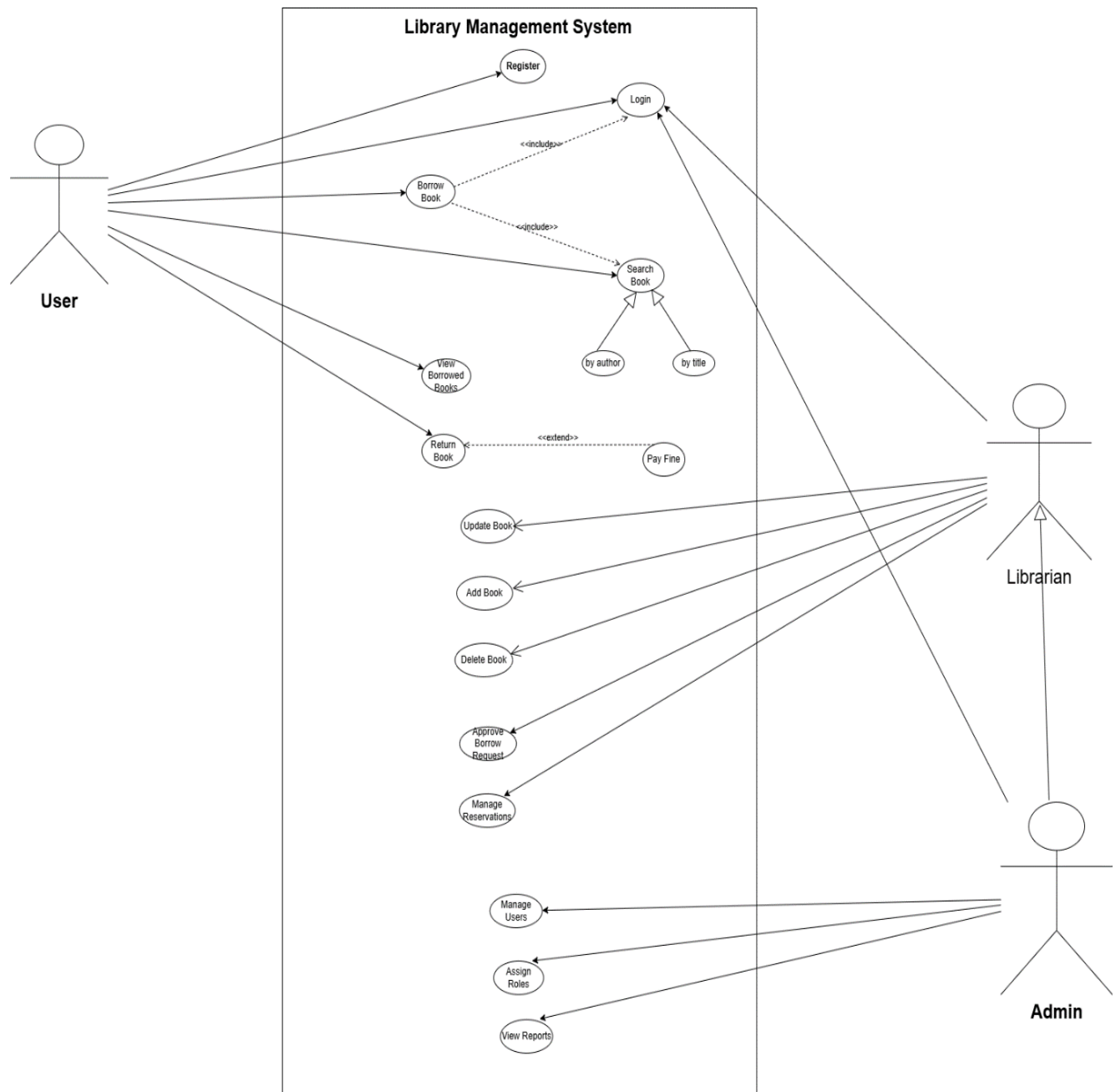
9.2 Deployment Steps

1. Build all microservices using Maven: `mvn clean package -DskipTests`
2. Build Docker images for each service using their respective Dockerfiles
3. Run `docker-compose up` to start all containers
4. Access the application at `http://localhost:3000`
5. Access the Eureka dashboard at `http://localhost:8761`
6. All API calls are routed through the API Gateway at `http://localhost:8080`

10. System Diagrams

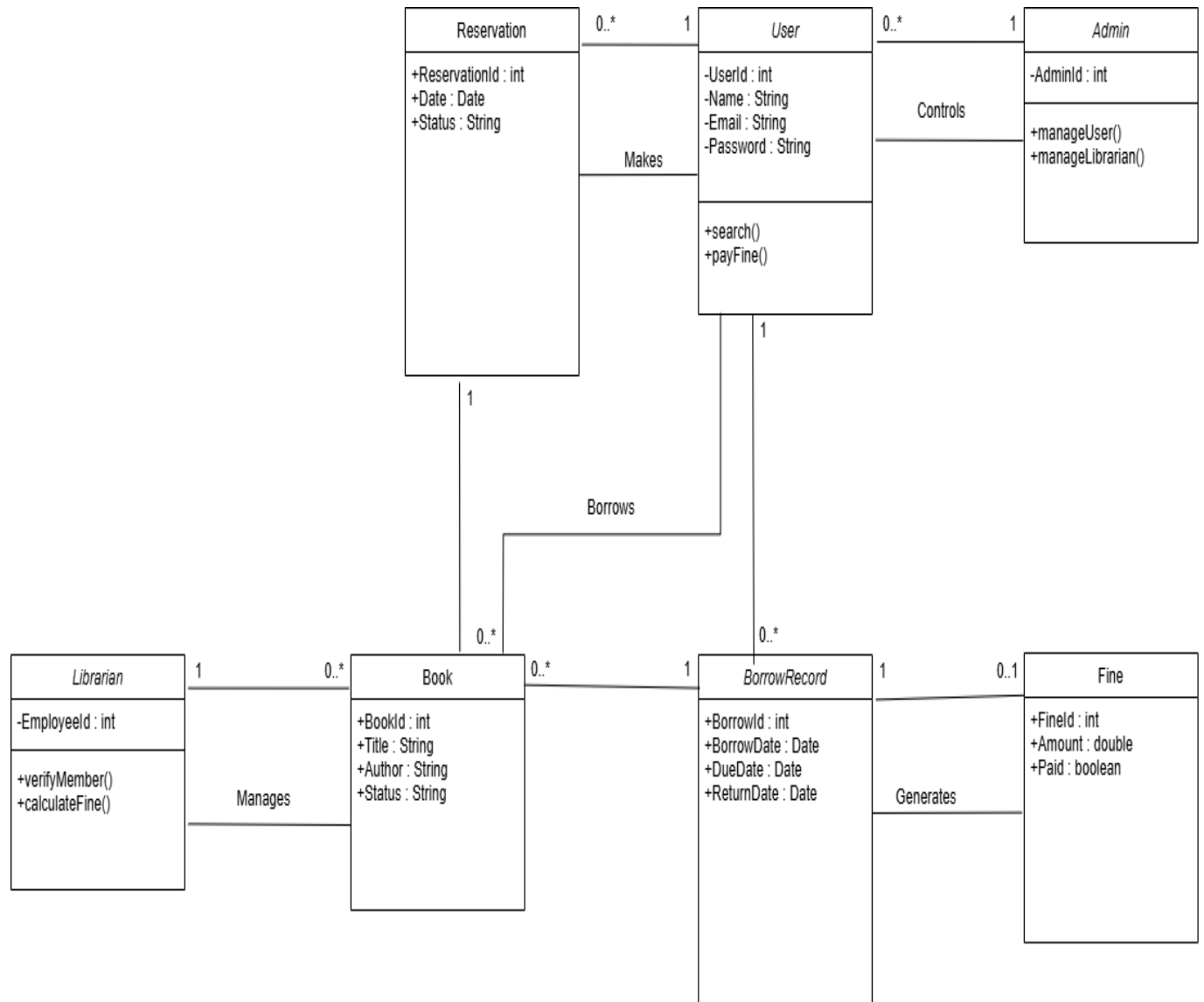
10.1 Use Case Diagram

The Use Case Diagram below illustrates the interactions between the three user roles (ADMIN, LIBRARIAN, and USER) and the main system functionalities.



10.2 Class Diagram

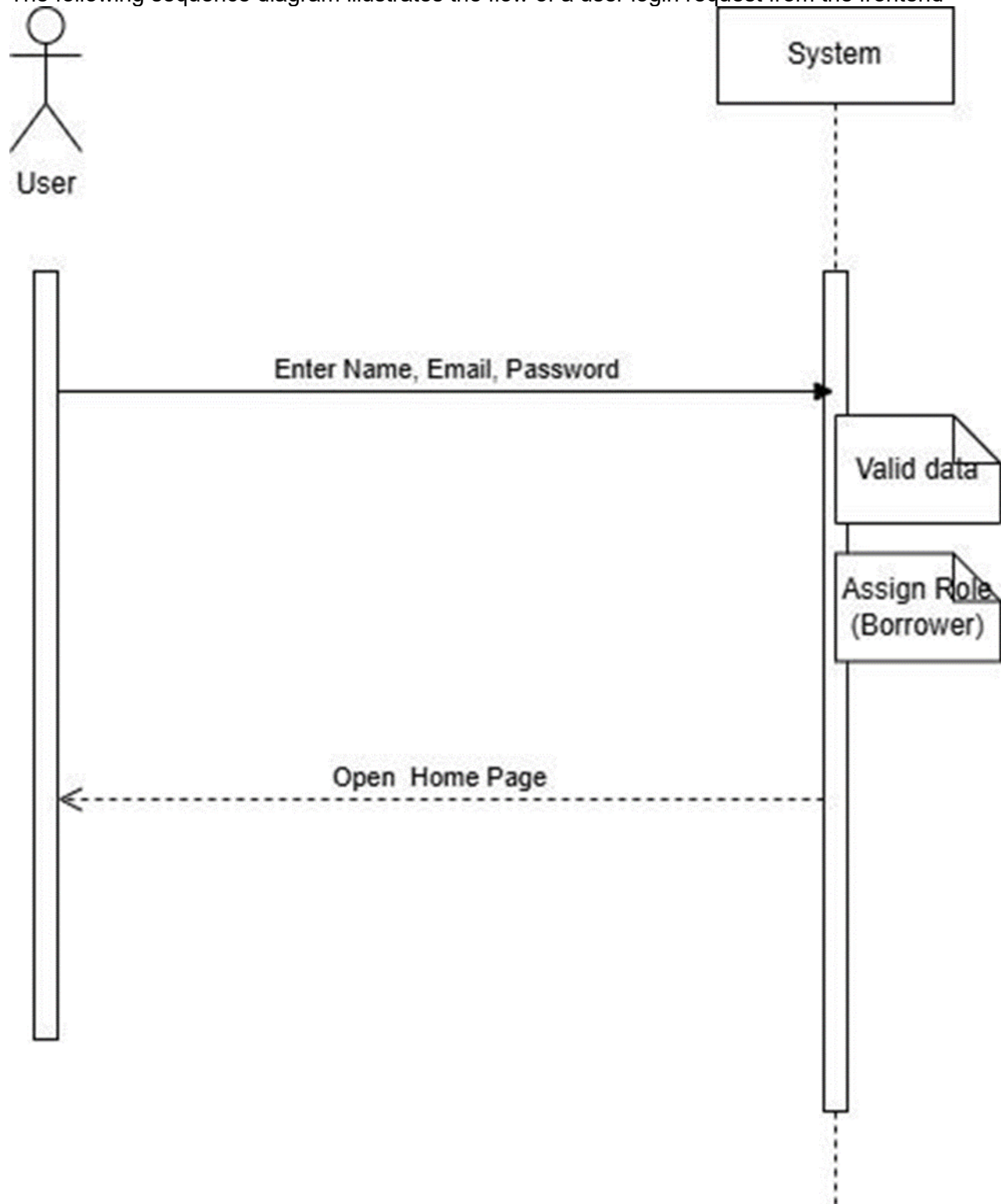
The Class Diagram below shows the main entity classes used across all microservices, including their attributes and relationships.



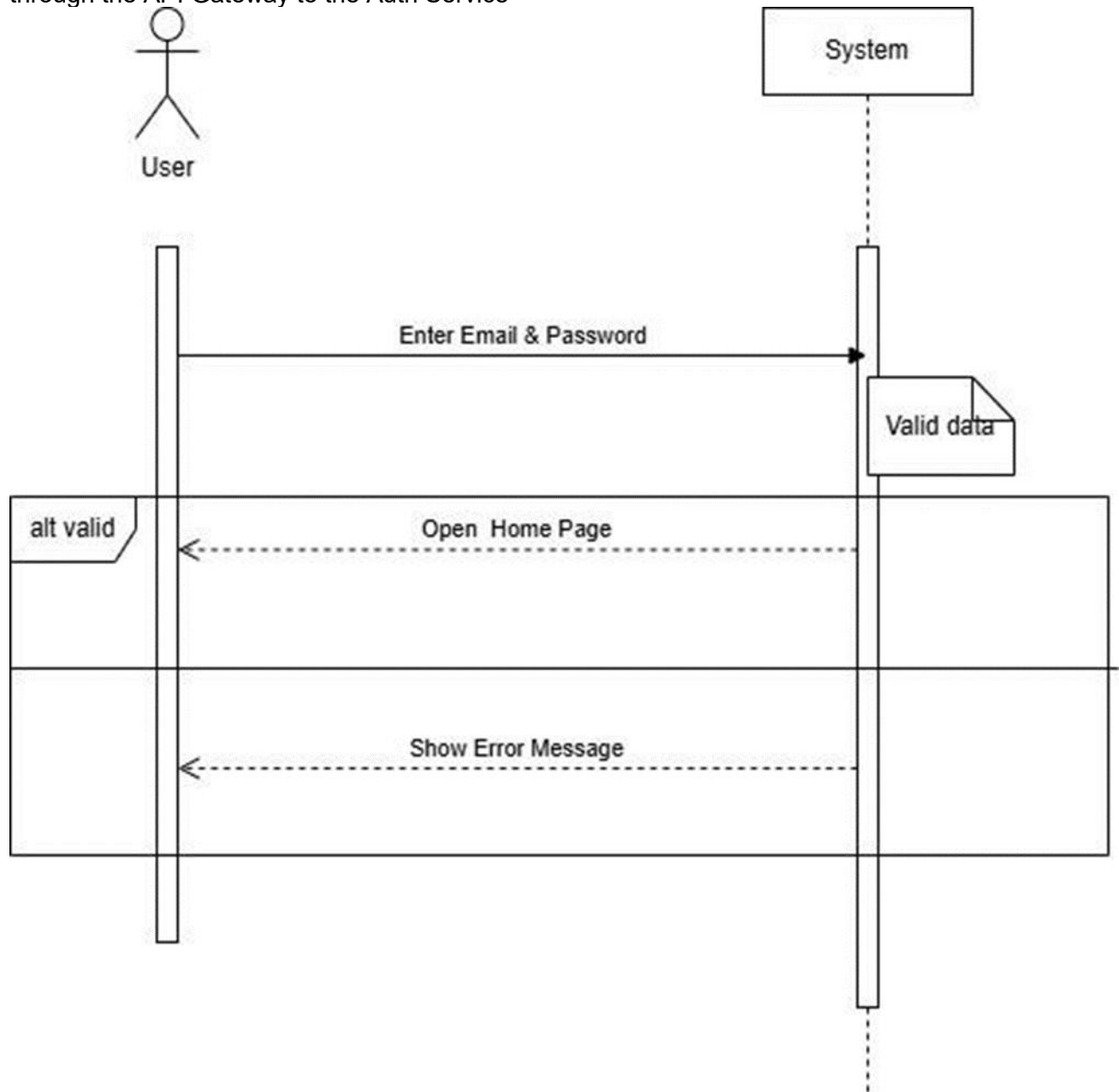
10.3 Sequence Diagrams

10.3.1 User Registration and Login Sequence

The following sequence diagram illustrates the flow of a user login request from the frontend

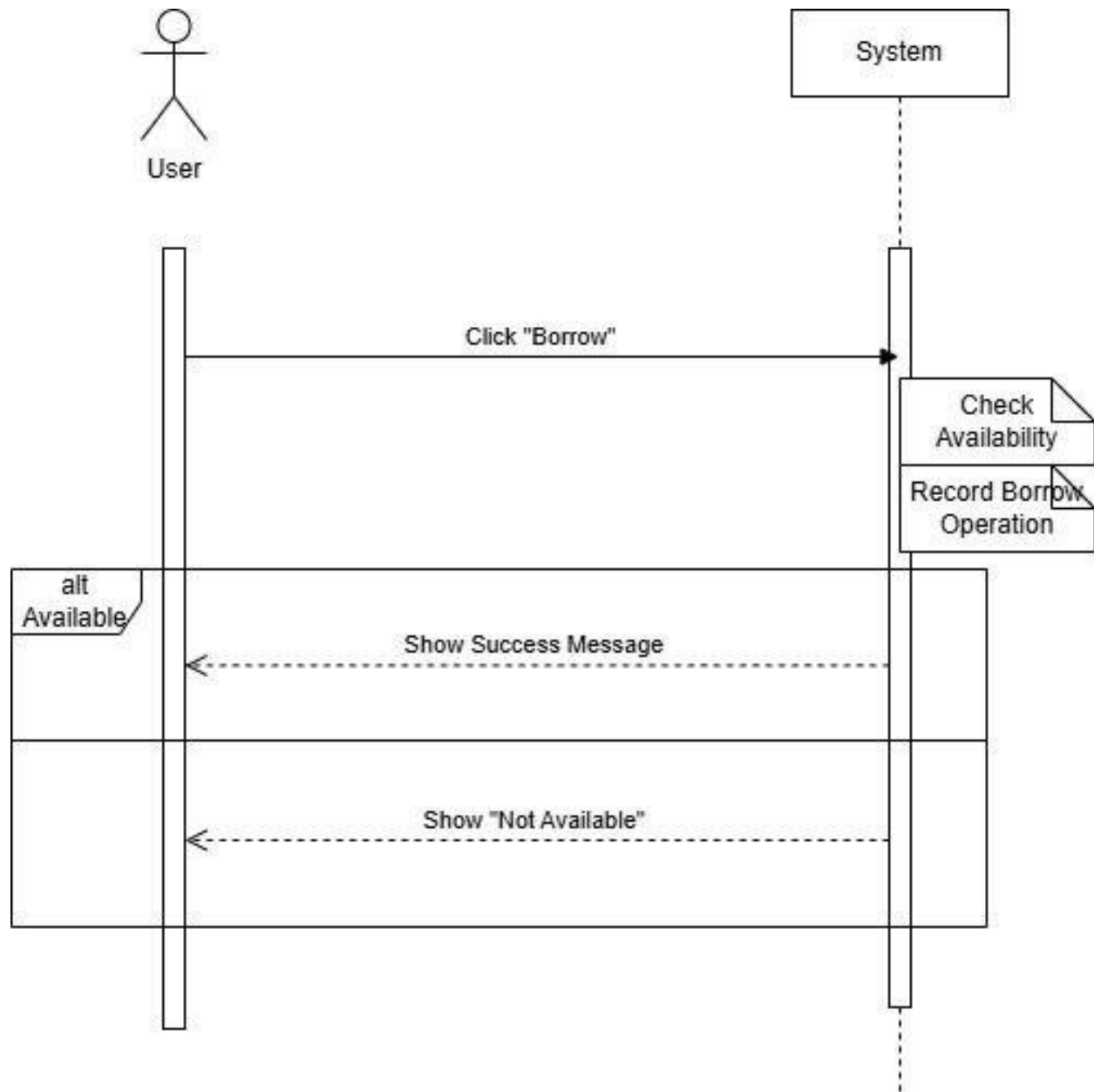


through the API Gateway to the Auth Service



10.3.2 Borrow Book Sequence

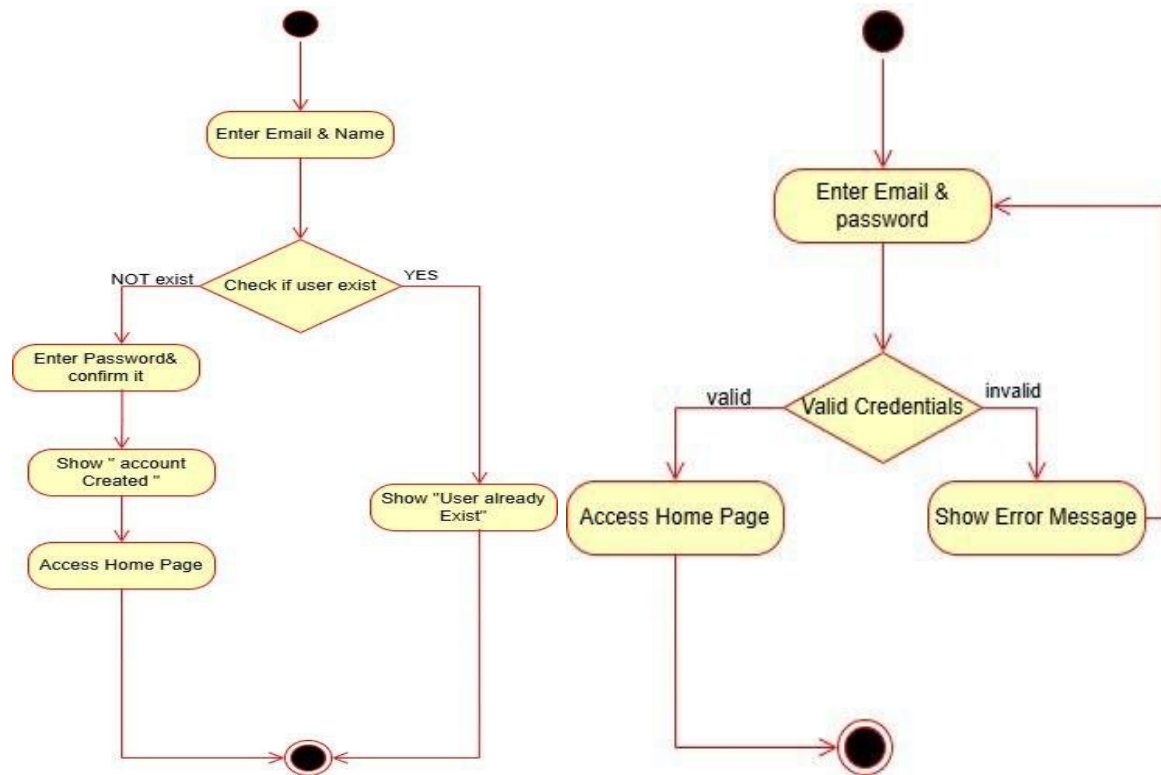
The following sequence diagram shows how a user borrows a book, including the inter-service communication between the Borrow Service and the Book Service.



10.4 Activity Diagrams

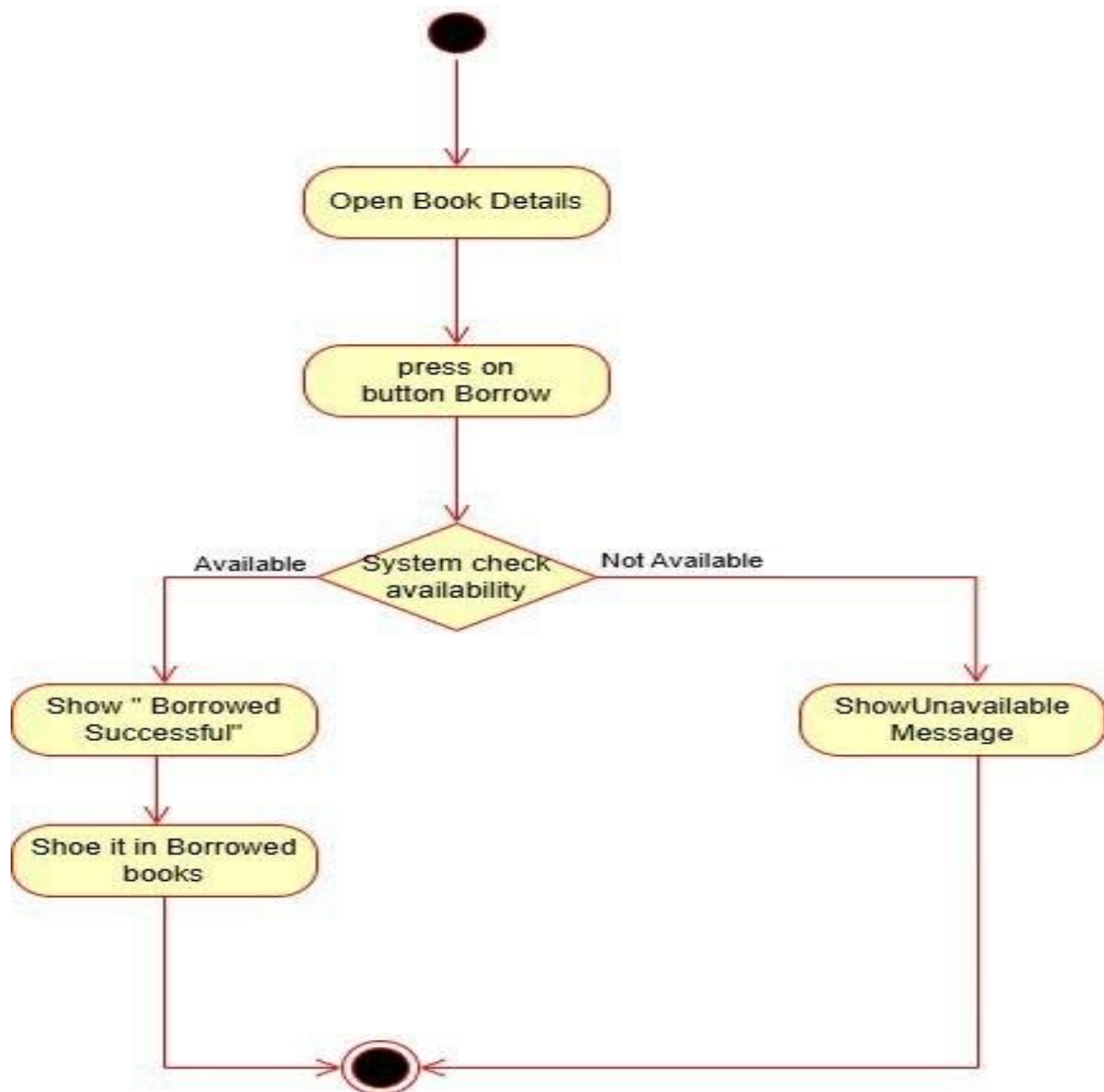
10.4.1 User Registration and Login Activity

The following activity diagram illustrates the complete flow of user registration and login, including validation steps and role assignment.

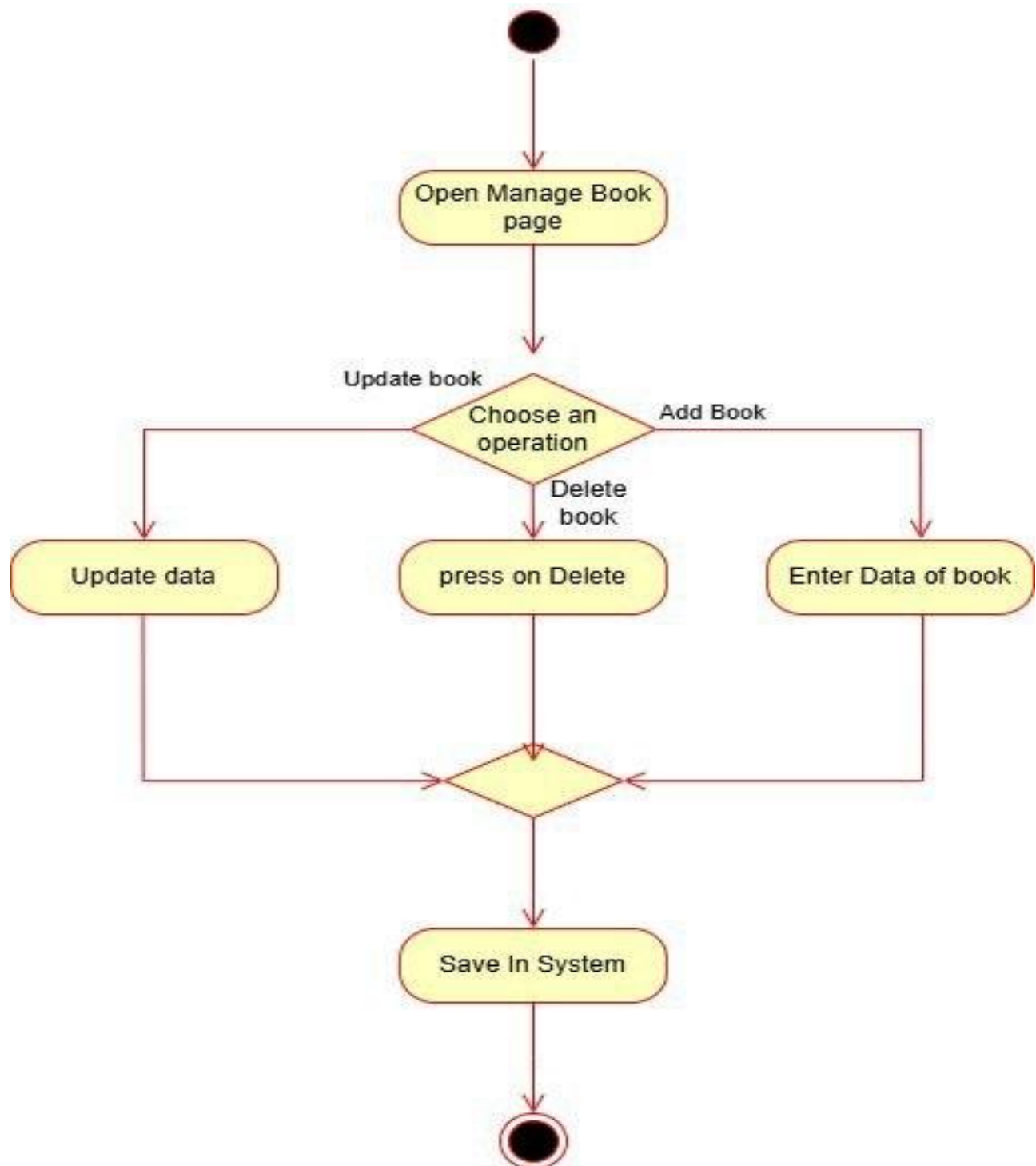


10.4.2 Book Borrowing Activity

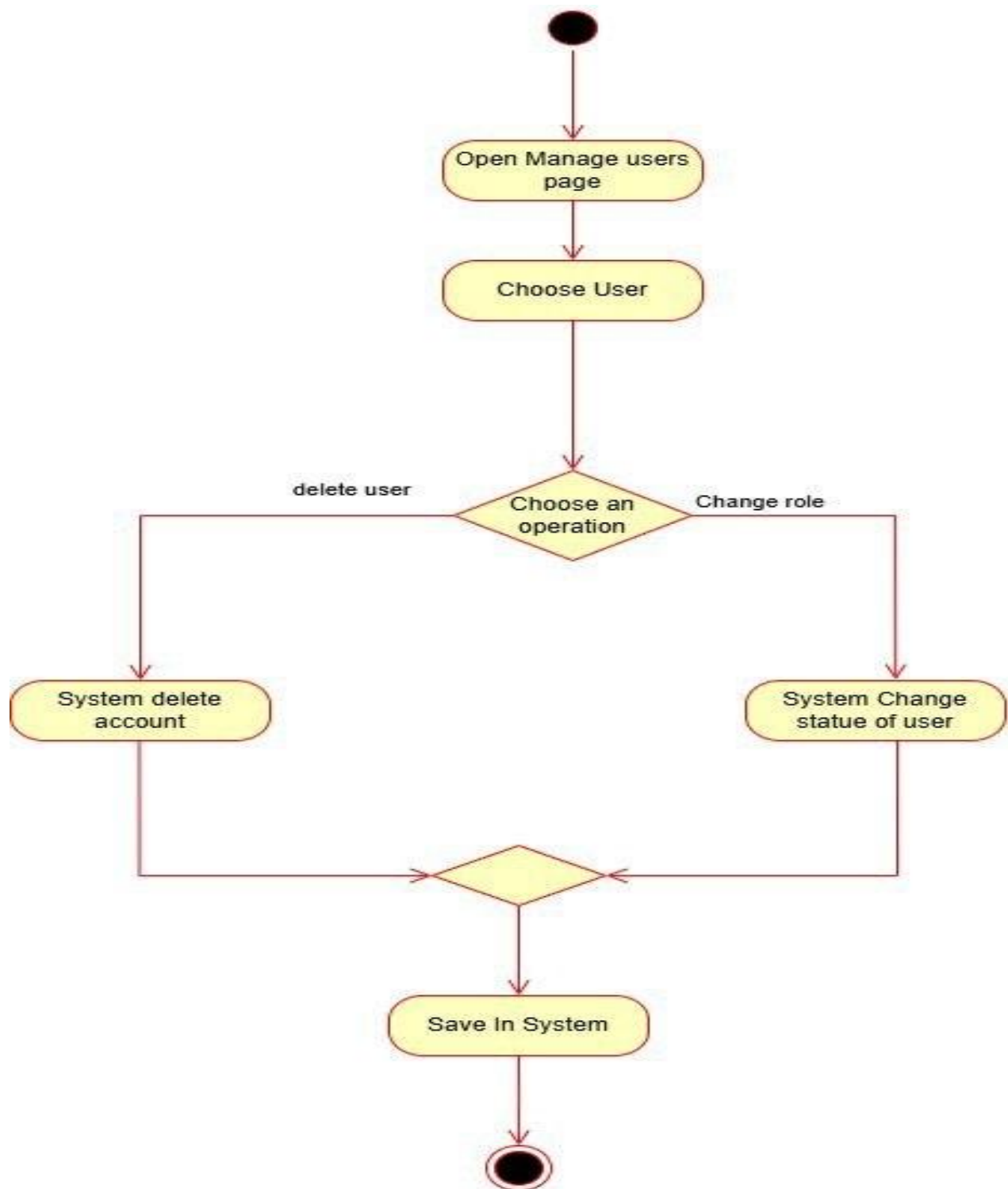
The following activity diagram shows the complete book borrowing process, from book selection to confirmation, including availability checks.



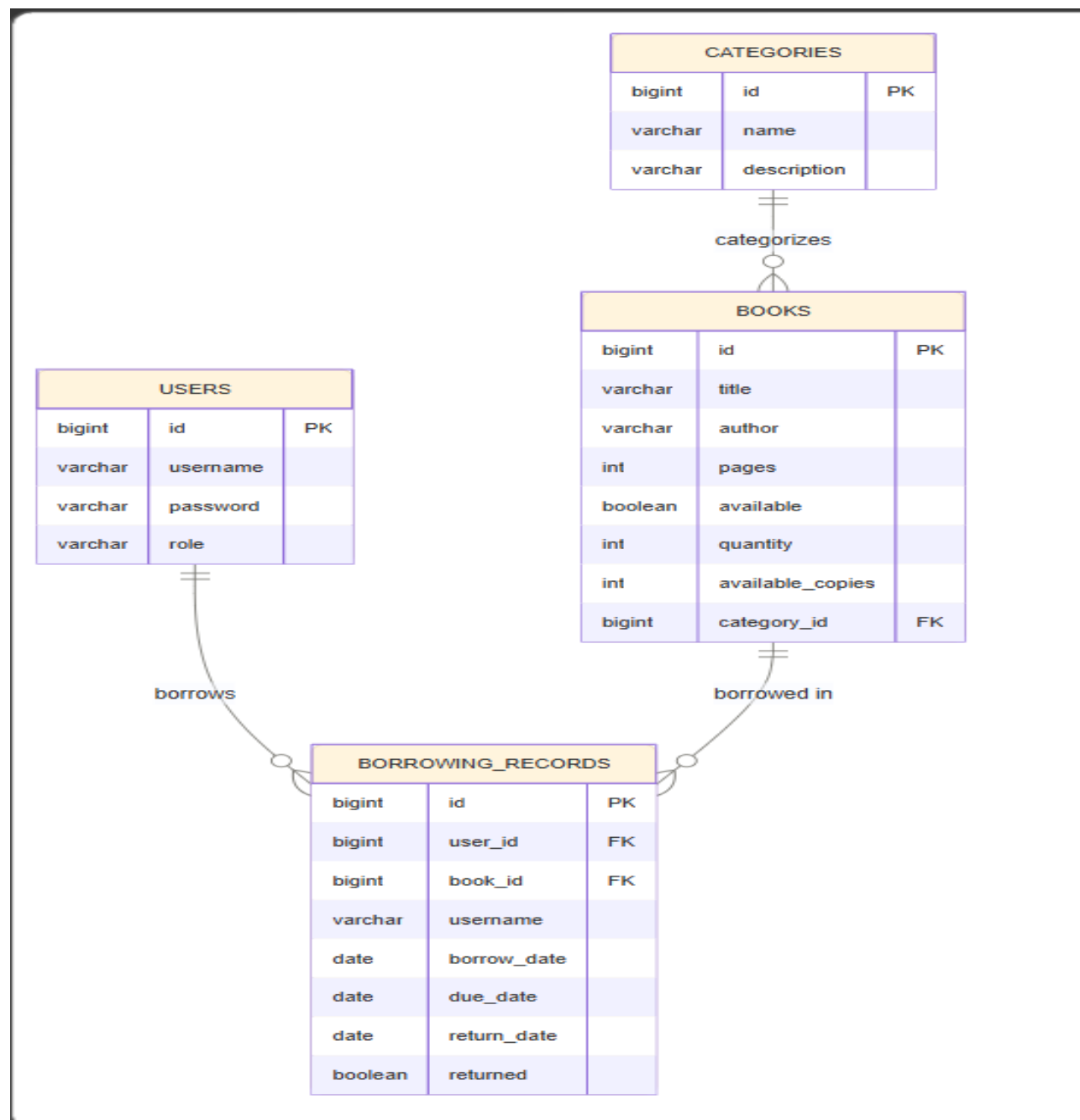
10.4.3 Manage Books Activity



10.4.4 Manage Users Activity



10.4.5 Entity Relationship Diagram (ERD)



11. Team Members

Name	Role	Responsibilities
Radwa Meshref	Frontend Developer	Project management, Frontend development, Borrow Service
Sondos Ahmed	Frontend Developer	Frontend routing, API integration, API Gateway, Book Service
Aya Taha	Backend Developer	Catalog Service development, Category management APIs
Mostafa Yasser	Backend Developer	React UI components, Microservices architecture, Docker configuration
Mohamed Mansour	Backend Developer	Auth Service, JWT implementation, Security configuration
Marwans Abdelfattah	Backend Developer	React UI components, Frontend development